



UNIVERSIDADE
FEDERAL DO CEARÁ
CAMPUS QUIXADÁ



DevOps

Profa. Lana Mesquita



DevOps

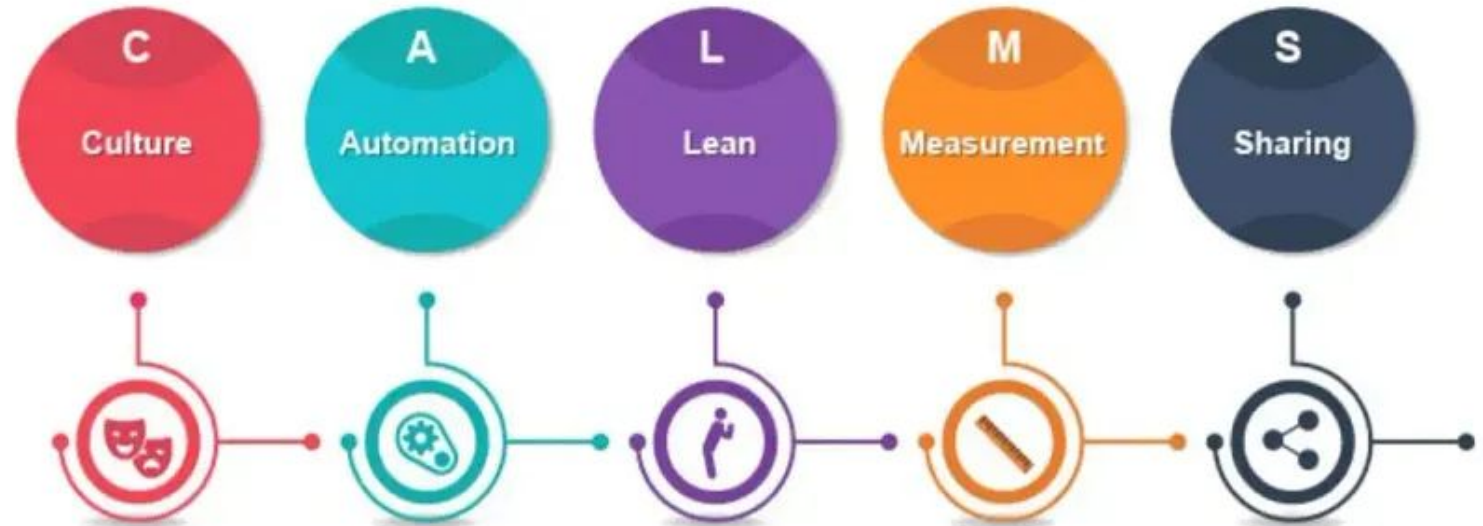
Cultura DevOps

DevOps é uma cultura e conjunto de práticas que visam integrar os times de desenvolvimento de software (Dev) e operações de infraestrutura (Ops).

O objetivo é **promover uma colaboração** mais estreita entre esses dois grupos, automatizar processos de desenvolvimento, testes e implantação, e criar um ambiente de entrega contínua e infraestrutura ágil.

DevOps busca **acelerar o ciclo de vida do desenvolvimento de software (SDLC)**, melhorar a qualidade do código e aumentar a eficiência operacional.

Cultura DevOps - CALMS



Salary by developer type

Senior Executives (\$139k), Engineering Managers (\$130k), and Financial Analysts (\$104k) have the highest median annual salaries globally.

The salary gap between the US and other countries is wide for higher paid roles. The median salary for an Engineering Manager in the US is \$200,000, compared to \$118,000 in Germany and just \$52,000 in India.

? What is your current total **annual** compensation (salary, bonuses, and perks, before taxes and deductions) in terms of your day-to-day currency? Please enter a whole number in the box below, without any punctuation. If you are paid hourly, please estimate an equivalent yearly salary. If you prefer not to answer, please leave the box empty.

All Respondents

Compare to 2024

United States

Germany

India

United Kingdom

France



SRE

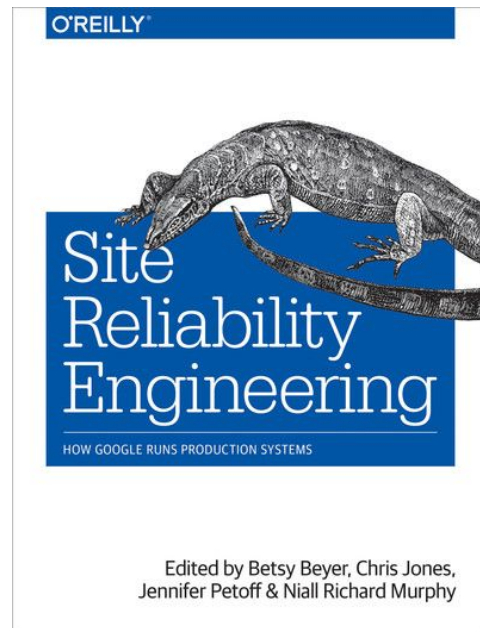
Fundamentos de SRE

Site Reliability Engineering (SRE) é um termo desenvolvido por Ben Treynor Sloss, vice-presidente de engenharia do Google criado por volta de **2003**.

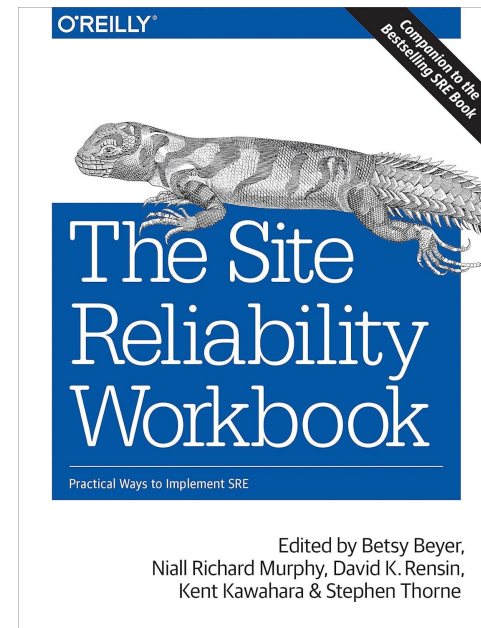
SRE é uma disciplina que incorpora aspectos da engenharia de software e os aplica a problemas de infraestrutura e operações.



Fundamentos de SRE



BEYER, Betsy; JONES, Chris; PETOFF, Jennifer; MURPHY, Niall Richard. **Site Reliability Engineering: How Google Runs Production Systems**. O'Reilly Media, 2016.



BEYER, Betsy; MURPHY, Niall Richard; RENSIN, David K.; KAWAHARA, Kent; THORNE, Stephen. **The Site Reliability Workbook: Practical Ways to Implement SRE**. O'Reilly Media, 2018.

Fundamentos de SRE

- Operações são um problema de software
- Orientação a Níveis de Serviço
- Redução de Toil
- Automação
- Reduzir o custo das falhas
- Responsabilidade compartilhada

DevOps x SRE

DevOps x SRE

Diferença DevOps x SRE:

- **DevOps é uma filosofia** sobre colaboração entre operações e desenvolvimento,
- **SRE é mais concreto** como uma função, focando na criação de **sistemas altamente confiáveis e escaláveis** através de automação e engenharia de software.
- SRE implementa a interface DevOps.
- SRE foi amplamente difundido por meio dos livros e adoção de empresas como o próprio Google, Netflix e Spotify.

DevOps x SRE

Devops é um conjunto de práticas, diretrizes e cultura projetada para quebrar silos em desenvolvimento, operações, arquitetura, rede e segurança.

SRE é uma disciplina que incorpora aspectos da engenharia de software e os aplica a problemas de infraestrutura e operações.

DevOps

Objetivo Principal: Promover uma cultura de colaboração entre equipes de desenvolvimento (Dev) e operações (Ops) para melhorar a entrega de software.

Foco: Abrange todo o ciclo de vida do desenvolvimento de software, desde a codificação até a operação.

Principais Práticas: Automação, colaboração, integração contínua, entrega contínua, monitoramento e feedback rápido.

Cultura: Destaca a importância da colaboração, comunicação e responsabilidade compartilhada entre as equipes de desenvolvimento e operações.

SRE

Objetivo Principal: Garantir a confiabilidade e disponibilidade dos sistemas de software, concentrando-se em práticas e ferramentas específicas para atingir esses objetivos.

Foco: Prioriza a confiabilidade, escalabilidade e desempenho dos sistemas em produção.

Principais Práticas: Definição de metas de serviço (SLOs), monitoramento proativo, gerenciamento de incidentes, automação de tarefas operacionais e engenharia de confiabilidade.

Cultura: Promove a automação, padronização e melhoria contínua dos processos operacionais para DevOps garantir a confiabilidade do sistema. SRE

DevOps x SRE

Semelhanças

- **Cultura de Colaboração:** Ambas enfatizam a importância da colaboração entre equipes de desenvolvimento e operações.
- **Automação:** Tanto DevOps quanto SRE valorizam a automação de processos repetitivos e tarefas operacionais.
- **Feedback Rápido:** Ambas as abordagens priorizam o feedback rápido e a melhoria contínua dos processos.

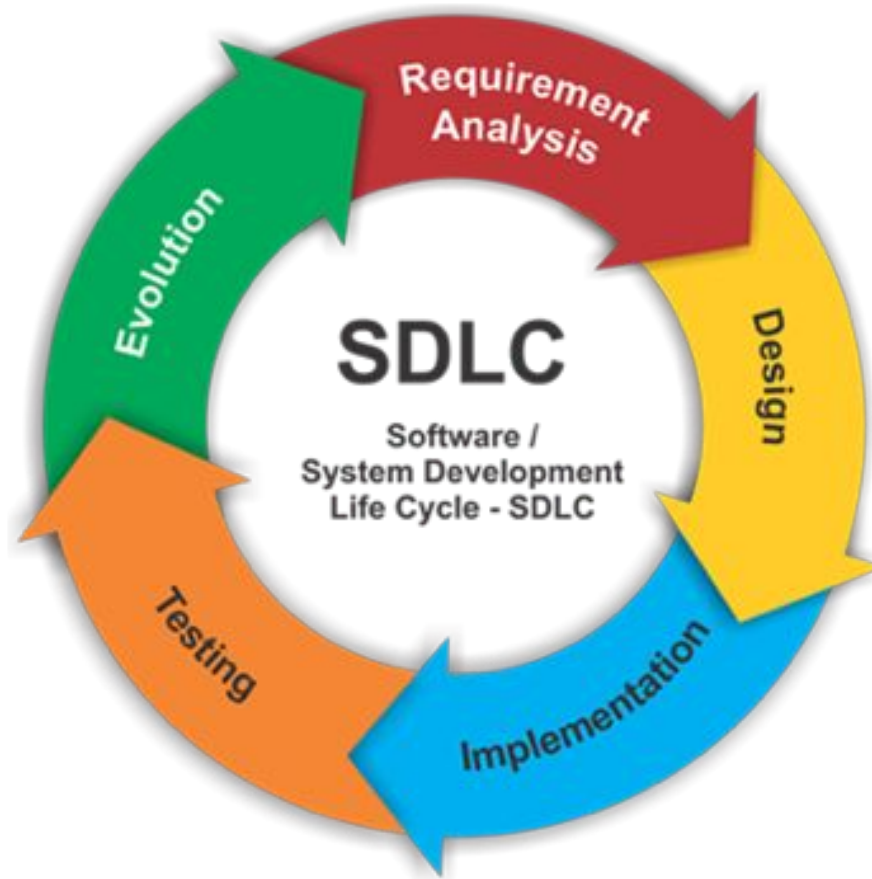
DevOps x SRE

Diferenças

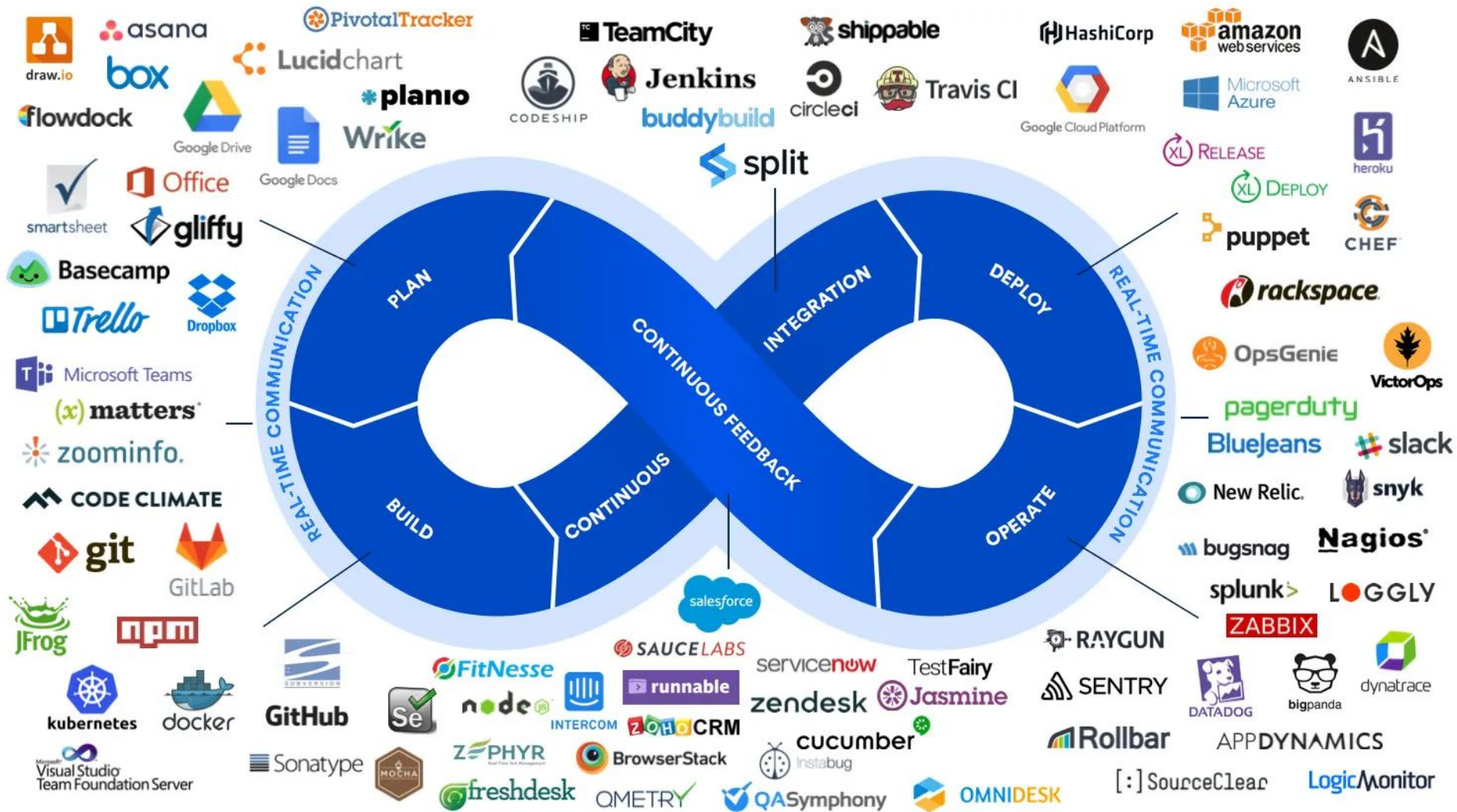
- **Escopo:** DevOps abrange todo o ciclo de vida do desenvolvimento de software, enquanto SRE se concentra especificamente na operação e confiabilidade dos sistemas.
- **Objetivos Primários:** DevOps visa melhorar a entrega de software, enquanto SRE visa garantir a confiabilidade e disponibilidade dos sistemas.
- **Práticas Específicas:** Embora compartilhem algumas práticas, como automação e colaboração, DevOps e SRE têm práticas específicas que se concentram em diferentes aspectos do desenvolvimento e operação de software.

Software Delivery Life Cycle (SDLC)

Software Delivery Life Cycle (SDLC)



Uma ruma de ferramenta



DevOps – Práticas

- Práticas de DevOps:
 - Infraestrutura como código (IaC)
 - Virtualização ou containerização
 - Integração contínua
 - Entrega contínua
 - Monitoramento

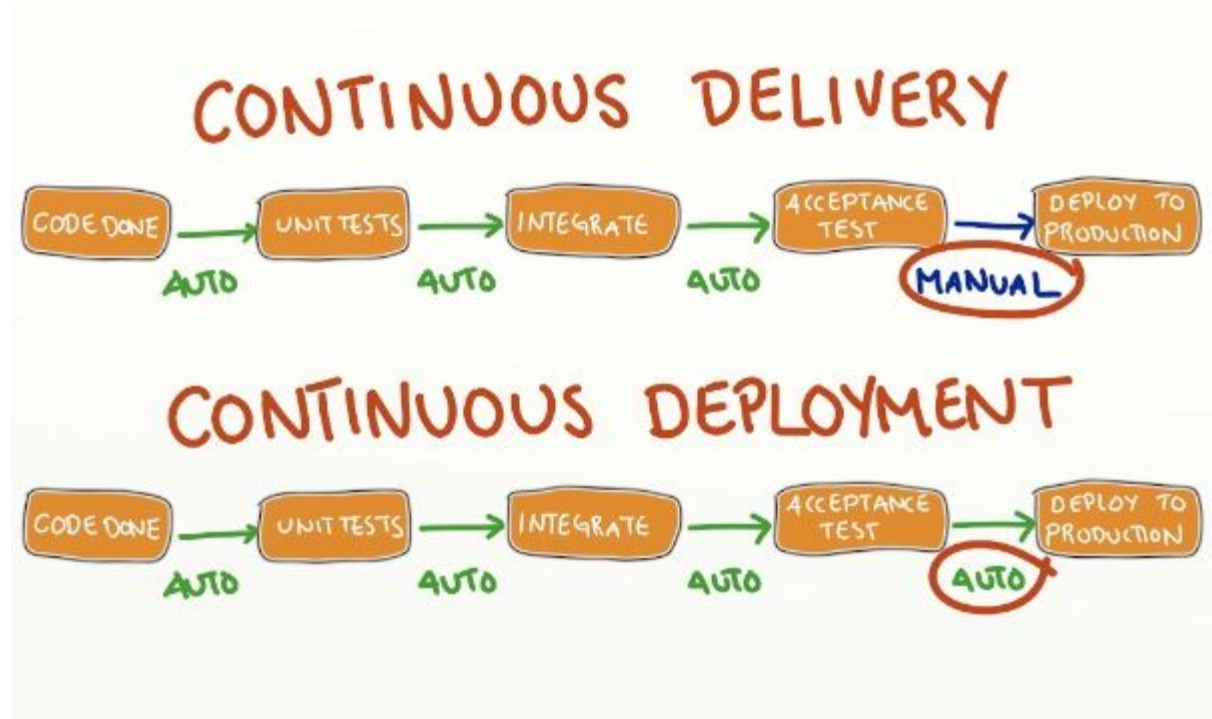
DevOps – Práticas

- Infraestrutura como código (IaC)
 - **DevContainer**
- Virtualização ou containerização
 - **Docker**
- Integração contínua
 - **GithubActions**
- Entrega contínua
 - Configuração **GithubActions + Docker**
- Monitoramento
 - ex. **SonarQube**



Deployment Contínuo

Entrega Contínua
(Continuous
Delivery)



Benefícios CI/CD

Os principais benefícios do CI/CD incluem:

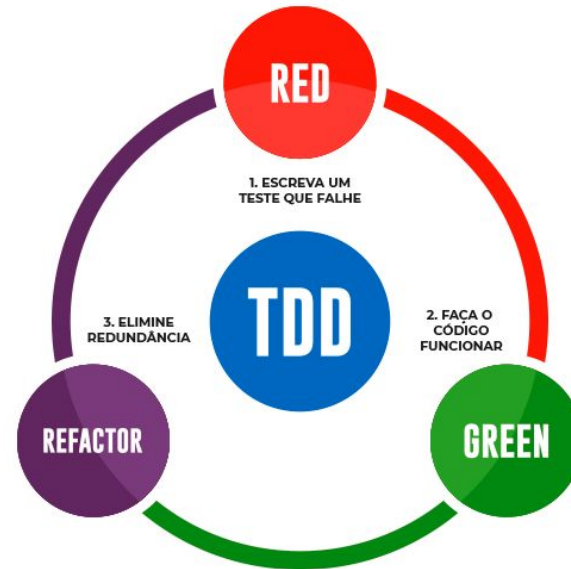
- Automação via Pipeline
- Entrega rápida e de valor
- Redução de riscos
- Melhoria da colaboração
- Implantação incremental
- Ambientes heterogêneos
- Rollbacks podem ser automatizados
- Testes em produção

Ferramentas CI/CD

- Github Action
- Gitlab CI
- Jenkins
- Circle CI
- Azure DevOps
- AWS CodePipeline

Testes de Software TDD

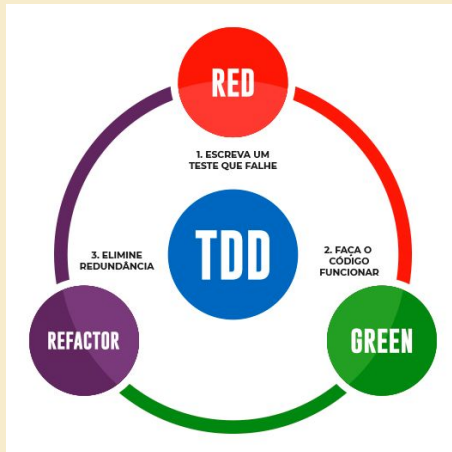
Test-Driven Development (TDD) é uma abordagem de desenvolvimento em que os testes são escritos antes mesmo do código de produção.



Testes de Software TDD

O ciclo típico de TDD envolve três etapas:

- **Red:** O desenvolvedor escreve um teste automatizado que define um comportamento desejado ou uma funcionalidade que ainda não existe no sistema. Este teste inicialmente falha, pois o código de produção correspondente ainda não foi implementado.
- **Green:** O desenvolvedor escreve o código mínimo necessário para fazer o teste passar. O objetivo aqui é implementar apenas o suficiente para satisfazer o teste recém criado.
- **Refactor:** Após o teste passar, o desenvolvedor pode refatorar o código para melhorar sua qualidade, sem alterar seu comportamento observável.



Testes de Software BDD

Behavior-Driven Development (BDD) é uma abordagem que se concentra no comportamento esperado do sistema em vez de nas implementações específicas.

Promove uma colaboração mais estreita entre desenvolvedores, testers e stakeholders do negócio, utilizando uma **linguagem comum para descrever o comportamento do sistema** em termos de cenários de usuário.

Testes de Software

Os cenários de usuário são escritos em uma **linguagem natural** compreensível por todas as partes interessadas, geralmente usando uma estrutura como Given-When-Then (Dado-Quando-Então).

Por exemplo:

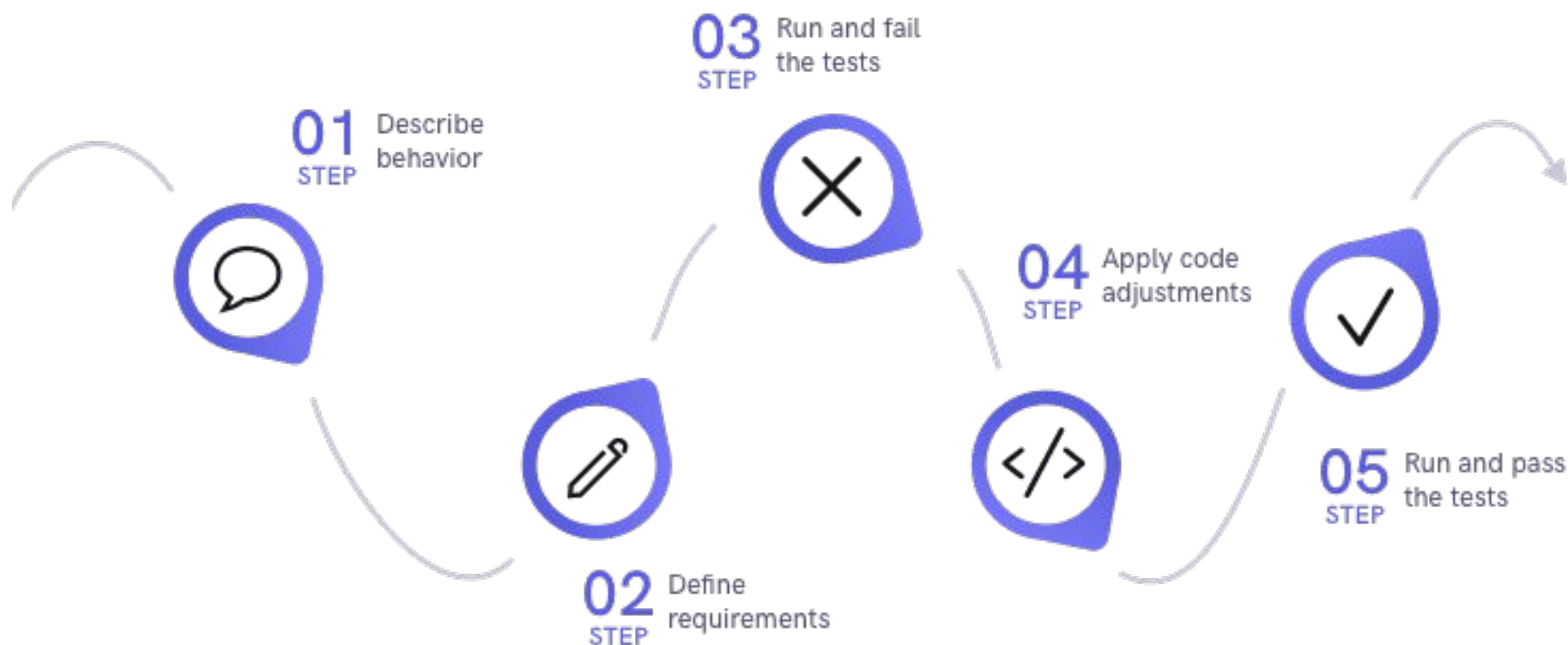
Dado que o usuário esteja na página de login

Quando ele inserir um nome de usuário e senha válidos

Então ele deverá ser redirecionado para a página inicial

Em seguida, esses cenários de usuário são traduzidos em testes automatizados, geralmente usando uma ferramenta de BDD, como Cucumber por exemplo.

Behavior Driven Development Cycle



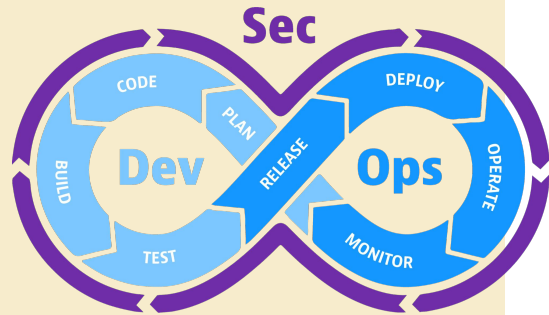
Abordagens de testes automatizados

Tipos de testes automatizados:

- Testes Unitários
- Testes Integrados
- Testes End to End
- Testes Regressivos
- Testes sintéticos
- Testes mutantes
- Testes Contínuos
- Matriz de testes

DevSecOps

DevSecOps



DevSecOps nada mais é que **incluir segurança no ciclo do desenvolvimento de software** e não mais como uma equipe ou departamento apartado onde tínhamos mais interação quando o software já estava em produção.

Essa abordagem traz muito **mais confiabilidade** ao produto pois trazemos etapas de segurança para fase inicial do desenvolvimento e garantindo que nosso software esteja sem vulnerabilidades críticas a serem exploradas, até porque não adianta um código funcional que não tem segurança.

SAST (Static Application Security Testing)

- É uma técnica de análise estática que examina o código-fonte ou o código compilado de um aplicativo em busca de vulnerabilidades de segurança.
- Ela procura por falhas de segurança, como vulnerabilidades de injeção de código (como SQL injection e XSS), uso inadequado de APIs e outras vulnerabilidades de código.
- O SAST é realizado durante a fase de desenvolvimento do software, geralmente como parte do processo de contínuos integration.

DAST (Dynamic Application Security Testing)

- É uma técnica de análise dinâmica que avalia o software em execução para identificar vulnerabilidades de segurança enquanto ele está sendo executado.
- DAST envolve pen tests automatizados ou manuais, onde é feito a simulação de ataques de um invasor externo.
- Ele procura por vulnerabilidades que podem ser exploradas quando o software está em execução, como configurações de servidor inadequadas, falhas de autenticação e autorização, entre outros.

IAST (Interactive Application Security Testing)

- É uma técnica que combina elementos de SAST e DAST.
- Ao contrário do SAST, que analisa o código-fonte em repouso, e do DAST, que testa o aplicativo em tempo de execução, o **IAST monitora a execução do aplicativo** durante os testes de segurança e examina o código-fonte em busca de vulnerabilidades enquanto o aplicativo está sendo executado.
- Isso permite uma análise mais precisa e contextualizada das vulnerabilidades, pois leva em consideração o contexto da execução do aplicativo.

Dúvidas?
Mandem por Discord!



Até a próxima aula!

Lana Mesquita

